

Linda Oraegbunam

Responsible AI & Model Governance Lead

GitHub: @obielin

Access Level: Maintainer

Reports To: @Goldokpa (Project Owner)

Project Duration: 3 Months

How This Role Fits You

Your FRSS fellowship, peer-reviewed publications in AI ethics, and open-source security tooling (skillguard) give you a governance lens that no one else on this team has. While others build models and pipelines, you ensure the outputs are trustworthy, explainable, and secure enough for NGOs and government agencies to act on.

Your skillguard project - detecting prompt injection and supply chain attacks in agentic AI - maps directly to securing ClimateVision's inference pipeline. Your agentsync work on multi-agent orchestration applies to building autonomous monitoring agents that track deforestation regions without human intervention.

Your HMRC experience operationalising 30+ governed KPIs and your Readrly.io MLOps work with fairness/bias evaluation frameworks mean you understand how to build auditability into production systems. That is exactly what ClimateVision needs: every prediction must be explainable, every model version auditable, and every regional disparity measurable.

Your Role on ClimateVision

You own the responsible AI, explainability, and governance layer - everything that turns black-box segmentation outputs into trustworthy, actionable intelligence for conservation stakeholders.

Core Responsibilities

- Build SHAP-based explainability for U-Net segmentation predictions - explain WHY each pixel was classified
- Implement bias and fairness audits across geographic regions (Amazon, Congo, Southeast Asia)
- Create anomaly detection for inference inputs and outputs - flag unusual predictions for human review
- Design model governance framework: version tracking, audit trails, compliance documentation
- Conduct AI security audit of FastAPI inference pipeline (OWASP Agentic Top 10, input validation)
- Build automated impact reports using LLM/NLP pipelines from model outputs
- Integrate responsible AI evaluation into CI/CD - automated fairness checks on every model release

Sprint Progress - April 2026

- PENDING: SHAP explainability integration for segmentation masks
- PENDING: Regional bias audit framework
- PENDING: Anomaly detection for inference pipeline
- PENDING: Model governance and audit trail system

- PENDING: AI security audit of API endpoints
- PENDING: LLM-based automated impact report generation

Your Codebase Ownership

You are the primary owner of the following files and directories:

src/climatevision/governance/	# PRIMARY OWNER - New module
explainability.py	# SHAP explainability for segmentation
bias_audit.py	# Regional fairness evaluation
anomaly_detector.py	# Input/output anomaly detection
audit_logger.py	# Model audit trails
__init__.py	
src/climatevision/security/	# PRIMARY OWNER - New module
api_security.py	# API input validation & sanitization
pipeline_guard.py	# Inference pipeline security checks
__init__.py	
src/climatevision/reports/	# PRIMARY OWNER - New module
llm_reporter.py	# LLM-based impact report generation
__init__.py	
scripts/	
audit_model.py	# Run full governance audit on a model
security_scan.py	# OWASP-style scan of API endpoints
notebooks/	
06_explainability.ipynb	# SHAP visualization notebook
07_bias_audit.ipynb	# Regional fairness analysis
08_security_audit.ipynb	# API security assessment

Your 3-Month Delivery Timeline

MONTH 1: Explainability & Bias (Weeks 1-4)

Week 1-2: SHAP Explainability

- Integrate SHAP (DeepExplainer) into U-Net forward pass for pixel-level attribution
- Build explainability.py - generate heatmaps showing which bands/regions drove predictions
- Create 06_explainability.ipynb with visual examples across all 3 analysis types
- Add /api/explain endpoint returning SHAP values for a given prediction

Week 3-4: Bias & Fairness Audit

- Build bias_audit.py - evaluate model performance disparity across regions
- Implement demographic parity and equalized odds metrics for geographic splits
- Create automated bias report generation (JSON + PDF)
- Write 07_bias_audit.ipynb demonstrating fairness evaluation

MONTH 2: Security & Anomaly Detection (Weeks 5-8)

Week 5-6: AI Security Audit

- Map ClimateVision API to OWASP Agentic Top 10 and MITRE ATLAS frameworks
- Build `api_security.py` - input sanitization, rate limiting, payload validation
- Implement `pipeline_guard.py` - detect adversarial inputs to inference pipeline
- Run `security_scan.py` against all endpoints and document findings

Week 7-8: Anomaly Detection

- Build `anomaly_detector.py` - isolation forest + statistical checks for predictions
- Flag predictions with confidence outside historical norms
- Create anomaly alert integration with Olufemi's alert system
- Add `/api/anomalies` endpoint for querying flagged predictions

MONTH 3: Governance & Reporting (Weeks 9-12)

Week 9-10: Model Governance

- Build `audit_logger.py` - immutable audit trail for every model version and prediction
- Integrate with MLflow for governance metadata tagging
- Create model card generator (automated from training config + evaluation metrics)
- Implement CI gate: block releases that fail fairness or security thresholds

Week 11-12: LLM Impact Reports & Documentation

- Build `llm_reporter.py` - generate natural-language impact reports from prediction data
- Integrate with Francis's carbon analytics for comprehensive stakeholder reports
- Write governance documentation and responsible AI deployment guide
- Final security sign-off and production readiness review

Your Git Workflow

```
# Create feature branches from develop
git checkout develop
git pull origin develop
git checkout -b feature/governance-shap-explainability

# Your branch naming convention:
feature/governance-*      (governance & explainability features)
feature/security-*        (security & audit features)
feature/reports-*         (LLM report generation)
fix/governance-*          (bug fixes)
```

All PRs go to the develop branch. Tag @edoh-Onuh for ML architecture reviews, Olufemi for API integration, and @franchise for analytics alignment.

Your Key Collaborators

- @edoh-Onuh (ML Lead) - His models are what you explain and audit. Coordinate on model architectures, checkpoint formats, and prediction outputs.
- Olufemi Taiwo (API Lead) - Your /api/explain and /api/anomalies endpoints extend his API. Align on schemas, auth, and rate limiting.
- @franchise (Analytics Lead) - His carbon metrics feed your LLM impact reports. Coordinate on data contracts and report templates.
- @cutewizzy11 (Full-Stack) - Frontend visualisations of SHAP heatmaps and anomaly dashboards need his React expertise.

Your Code Pipeline

Your pipeline covers explainability generation, bias evaluation, security scanning, and governance reporting.

Step 1: Environment Setup

```
git clone https://github.com/Climate-Vision/ClimateVision.git
cd ClimateVision
pip install -r requirements.txt
pip install shap mlflow transformers torch

# Verify dependencies
python -c "import shap, transformers, torch; print('Governance stack ready')"
```

Step 2: Generate SHAP Explanations

```
# Run SHAP explainability on a prediction mask
python - <<'EOF'
from climatevision.governance.explainability import explain_prediction
explanation = explain_prediction(
    model_path='models/unet_deforestation.pth',
    image_path='data/test/amazon_tile.tif',
    analysis_type='deforestation'
)
print(f"Top contributing bands: {explanation['top_bands']}")
print(f"Spatial attribution saved to: {explanation['heatmap_path']}")
EOF
```

Step 3: Run Bias Audit

```
# Evaluate fairness across geographic regions
python - <<'EOF'
from climatevision.governance.bias_audit import run_bias_audit
report = run_bias_audit(
    model_path='models/unet_deforestation.pth',
    regions=['amazon', 'congo', 'southeast_asia'],
    metric='equalized_odds'
)
print(f"Fairness score: {report['score']:.3f}")
print(f"Disparity detected: {report['disparity_regions']}")
EOF
```

Step 4: Security Scan

```
# Run OWASP-style scan on API endpoints
python scripts/security_scan.py --target http://localhost:8000

# Output: security_report.json with findings and remediation
```

Step 5: Generate LLM Impact Report

```
# Generate natural-language report from prediction data
python - <<'EOF'
from climatevision.reports.llm_reporter import generate_impact_report
report = generate_impact_report(
    run_id=12345,
    region='amazon',
    include_shap=True,
    include_carbon=True
)
print(report['summary'])
# -> 'The model detected 1,247 hectares of deforestation with 94% confidence...'
EOF
```

Step 6: Commit & Push Your Work

```
# Switch to your git identity (create one if needed)
git config user.name 'Linda Oraegbunam'
git config user.email 'linda@placeholder.com'

git checkout develop && git pull origin develop
git checkout -b feature/governance-shap-explainability

git add src/climatevision/governance/
git add notebooks/06_explainability.ipynb
git commit -m "feat(governance): add SHAP explainability for segmentation predictions"

git push origin feature/governance-shap-explainability
```